

SENSORES, ATUADORES E ARMAZENAMENTO

Prof. Lucas Rafael Filipak

TEMA 1 – INTERNET DAS COISAS

Iniciamos esta aula com esse tema porque ele está estreitamente relacionado com essa disciplina. A internet das coisas, ou *Internet of things* (IoT), descreve um cenário em que vários objetos do dia a dia podem se comunicar e estar conectados à internet. As TVs ou videogames são exemplos de equipamentos que passaram do mundo *offline* para o mundo online. Pense em outros objetos que também podem sofrer tal migração: uma geladeira, câmeras de segurança, fechaduras, lâmpadas etc.

A ideia da IoT é facilitar a vida dos usuários. Gogoni (2018) relatou algumas tarefas que servem como exemplo:

- Um aspirador de pó robô pode ser programado para limpar a casa depois da hora de dormir;
- As lâmpadas da casa podem emitir luzes em tons específicos durante vários momentos do dia, ou se apagarem quando todos saírem;
- O ar-condicionado pode ser ativado cinco minutos antes de você chegar, deixando o ambiente na temperatura correta;
- Um fogão seria capaz de cortar o fornecimento de gás e avisar a companhia fornecedora ao detectar um vazamento;
- Freezers comerciais avisariam o fabricante em caso de defeito, evitando a perda de comida, vacinas ou outros elementos perecíveis ou pouco duráveis sem a devida refrigeração;
- Tratores automatizados seriam capazes de fazer o trabalho de um funcionário, mesmo à noite, por meio de dados via satélite para evitar desperdício e utilizando a rede apenas quando necessário;
- Hospitais podem utilizar equipamentos capazes de coletar dados armazenados em *smartwatches*, pulseiras inteligentes e outros dispositivos vestíveis que monitorem os dados vitais do paciente, otimizando o atendimento e facilitando o diagnóstico.

A internet das coisas está no início da sua enorme caminhada. Muitos projetos são bem simples de serem implementados, outros são mais complexos, mas o mais importante é que existe uma enorme comunidade que pesquisa e disponibiliza soluções resolvidas.

Podemos citar como exemplos alguns pequenos projetos que podem ser implementados no dia a dia de um usuário. Alguns deles nós desenvolveremos juntos nesta disciplina.

- Automação do sistema de luzes, ligando automaticamente quando o ambiente ficar escuro e desligando quando o ambiente ficar claro.
- Sistema para regar o jardim. Um sensor que fica debaixo da terra checa o nível de umidade e, dependendo da programação, abre ou fecha a torneira de irrigação.
- Dependendo da temperatura, o sistema liga ou desliga automaticamente, mantendo a temperatura sempre agradável.

Para finalizar esse pequeno tema, que tem como objetivo aguçar a curiosidade dos estudantes, citamos Gogoni (2018), ao afirmar:

A Internet das Coisas é também essencial ao conceito de Cidades Inteligentes, projetos urbanísticos que levam em conta sustentabilidade, eficiência e qualidade de vida. Num futuro não muito distante teremos veículos autônomos, semáforos que abrem e fecham de acordo com o tráfego e não em períodos pré-programados, sistemas de reaproveitamento de água, ciclovias inteligentes e etc.

TEMA 2 – INTRODUÇÃO AO ARDUINO

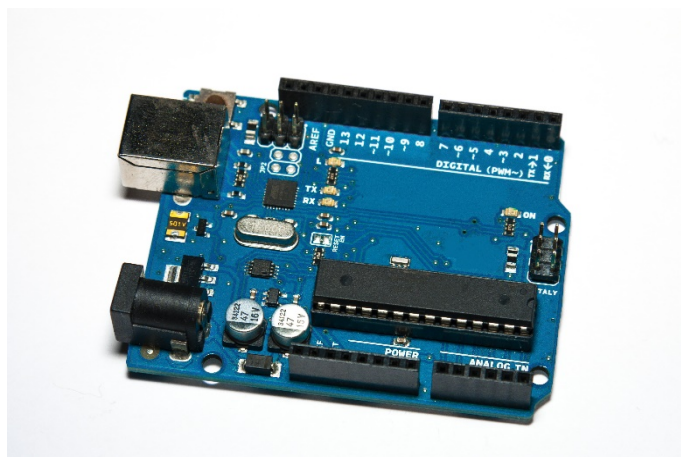
O Arduino é um microcontrolador desenvolvido na Itália, no ano de 2005, por um grupo de 5 pesquisadores¹. Trata-se de uma placa de prototipagem que apresenta hardware e software com tecnologia aberta. Massimo Banzi, um dos criadores do Arduino, declarou em uma palestra (TED) que:

o objetivo era elaborar um dispositivo que fosse ao mesmo tempo barato, funcional e fácil de programar, sendo dessa forma acessível a estudantes e projetistas amadores. Além disso, foi adotado o conceito de hardware livre, o que significa que qualquer um pode montar, modificar, melhorar e personalizar o Arduino, partindo do mesmo hardware básico. (Banzi, 2012)

A figura 1 representa um Arduino do modelo UNO. Existem diversos modelos de **placas Arduino**, mas essa é a mais popular, desenvolvida por Arduino.cc.

¹ Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino e David Mellis.

Figura 1 – Placa Arduino

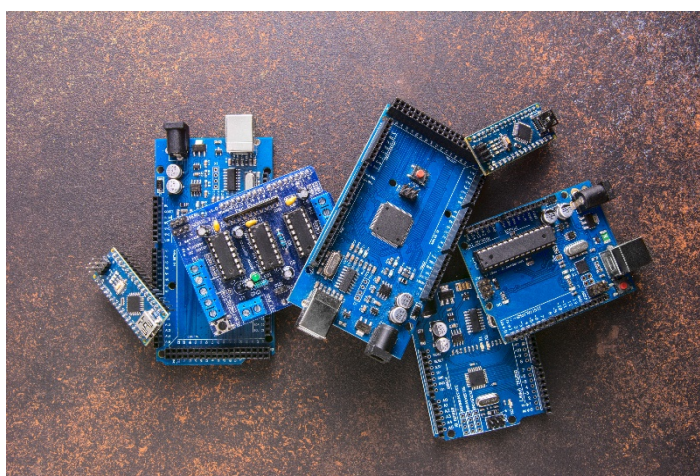


Fonte: Sirio Brunialti/Shutterstock.

Acessando o site² do Arduino, é possível baixar o esquema elétrico e todos os arquivos do projeto para edição. Apesar de haver diferentes modelos de Arduinos, os projetos programados em um modelo podem ser facilmente adaptados para outro, respeitando as particularidades de cada placa.

O tipo de placa a ser utilizada dependerá do tipo de projeto a ser executado, variando, basicamente, em número de portas e memória interna de armazenamento. A Figura 2 ilustra alguns tipos de Arduinos conhecidos, pois se trata de um produto com código aberto, várias empresas produzem as suas próprias placas.

Figura 2 – Tipos de placas de Arduino



Crédito: Sergey Privalov/Shutterstock.

² <www.arduino.cc>

Na Figura 2, podemos ver somente a diferença no visual (design) de cada placa. Para ter uma melhor avaliação das placas, observe a Figura 3.

Figura 3 – Tabela de comparação de tipos de Arduino

	UNO	MEGA 2560	LEONARDO	DUE	ADK	NANO	PRO MINI	ESPLORA
Microcontrolador	ATmega328	ATmega2560	ATmega32u4	AT91SAM3X8E	ATmega2560	ATmega168 (versão 2.x) ou ATmega328 (versão 3.x)	ATmega168	ATmega32u4
Portas digitais	14	54	20	54	54	14	14	-
Portas PWM	6	15	7	12	15	6	6	-
Portas analógicas	6	16	12	12	16	8	8	-
Memória	32K (0,5K usado pelo bootloader)	256K (8K usado pelo bootloader)	32K (4K usado pelo bootloader)	512K disponível para aplicações	256K (8K usado pelo bootloader)	16K (ATmega168) ou 32K (ATmega328) (bootloader: 2K)	16K (2K usado pelo bootloader)	32K (4K usado pelo bootloader)
Clock	16Mhz	16Mhz	16Mhz	84Mhz	16Mhz	16Mhz	8Mhz (modelo 3.3v) ou 16Mhz (modelo 5v)	16Mhz
Conexão	USB	USB	Micro USB	Micro USB	USB	USB Mini-B	Serial/Módulo USB externo	Micro USB
Conector para alimentação externa	Sim	Sim	Sim	Sim	Sim	Não	Não	Não
Tensão de operação	5V	5V	5V	3.3V	5V	5V	3.3 ou 5V, dependendo do modelo	5V
Corrente máxima portas E/S	40mA	40mA	40mA	130mA	40mA	40mA	40mA	-
Alimentação	7-12Vdc	7-12Vdc	7-12Vdc	7-12Vdc	7-12Vdc	7-12Vdc	3.3-12V (modelo 3.3v) ou 5-12V (modelo 5v)	5V

Fonte: Thomsen, 2014.

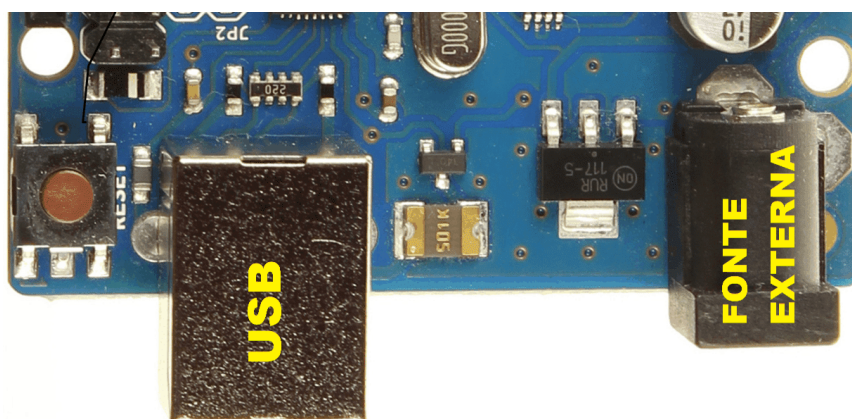
Para maiores informações sobre as placas, você deve acessar o site da empresa que a produziu, contudo, a tabela da Figura 3 traz as principais especificações de cada placa.

Para estas aulas, o Arduino UNO foi o escolhido para os exemplos, pois ele é desenvolvido pelo site Arduino.cc.

TEMA 3 – ALIMENTAÇÃO ARDUINO

O Arduino, como qualquer eletrônico, deve receber uma fonte de alimentação. Note que, na Figura 4, há duas conexões responsáveis pela alimentação da placa.

Figura 4 – Alimentação do Arduino



Fonte: Souza, 2013.

A primeira entrada representada na Figura 4 tem formato quadrado e é utilizada para conectar o cabo USB no Arduino. O cabo USB serve para passar a programação desenvolvida em um computador (IDE) para a memória interna do Arduino. Quando a placa está conectada ao computador via cabo USB, não é preciso fazer qualquer tipo de alimentação extra, pois o cabo USB faz as duas funções, a comunicação com o computador e a alimentação com energia. Com o cabo USB conectado, sua programação é feita via IDE³, que pode ser baixada direto do site oficial do Arduino (<www.arduino.cc>), ou também é possível utilizar uma ferramenta de simulação on-line (<http://create.arduino.cc>) do próprio Arduino.

Para programar um Arduino, usa-se uma linguagem baseada em C/C++. Muitos materiais afirmam que a linguagem de programação do Arduino é a Wiring, no entanto, Wiring é uma plataforma também de prototipagem eletrônica composta de uma linguagem de programação e uma interface de desenvolvimento (as famosas IDEs), cuja essência é C++.

A linguagem C é considerada de baixo nível, isto é, próxima da linguagem dos microcontroladores e processadores, em que o programador deve controlar os bits, manipular a memória e executar instruções do processador. A linguagem C++ é considerada de alto nível, ou seja, seu nível de abstração é maior e ela está mais próxima da linguagem humana do que da linguagem dos computadores, não precisando fazer manipulações de memória e dos recursos de hardware diretamente, mas sim por meio de bibliotecas e recursos previamente disponíveis que deixam a linguagem mais transparente para o programador. A linguagem

³ *Integrated Development Environment* ou “Ambiente de Desenvolvimento Integrado”.

Arduino utiliza uma série de bibliotecas e recursos para que o usuário não precise manipular bits e registradores do microcontrolador.

A segunda entrada é a fonte de alimentação externa, com o formato arredondado (conector Jack), e serve para conectar uma fonte externa de energia ao Arduino. A fonte externa vai fornecer a energia necessária para o funcionamento do Arduino, pois, depois de programado, o Arduino pode ser usado de forma independente, ou seja, pode ser colocado em um protótipo, um robô, nas luzes de casa, para acionar um ventilador automaticamente quando estiver calor ou em qualquer outro projeto. A alimentação via fonte externa deve estar entre 6v e 20v, mas o ideal é alimentar com uma bateria de 12v, pois a tensão de funcionamento da placa é de 5v e pode ficar instável se for alimentado com menos de 7v. Além disso, o regulador de tensão da placa pode sobreaquecer se for alimentado com uma tensão acima de 12v.

Depois que a placa for energizada, ela pode fornecer (distribuir) diversos valores de tensão para os componentes utilizados no projeto. Observe, na Figura 5, quais são os conectores de alimentação do Arduino UNO.

Figura 5 – Conectores de alimentação



Fonte: Souza, 2013.

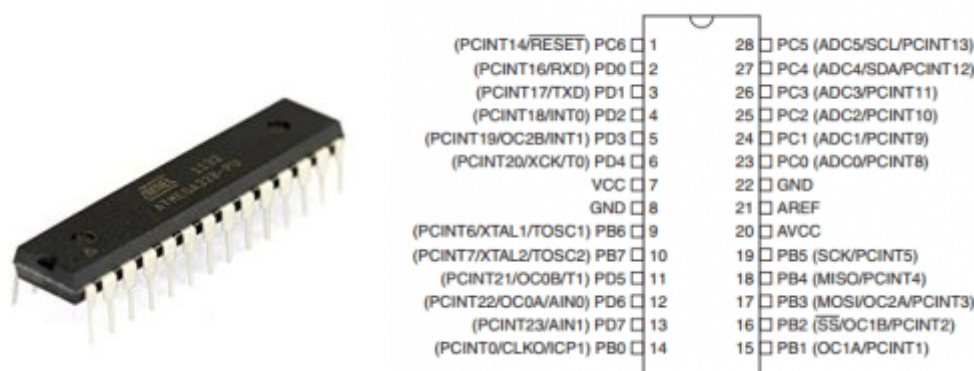
- **IOREF** – indica a tensão em que o Arduino trabalha. O UNO trabalha com 5V. O seu objetivo é permitir que *shields* conectadas a Arduinos de 3V detectem a tensão na qual o Arduino opera.
- **RESET** – pino conectado a pino de RESET do microcontrolador.
- **3,3 V** – Fornece tensão de 3,3V para alimentação de *shield* e módulos externos. Corrente máxima de 50 mA.
- **5 V** – Fornece tensão de 5V para alimentação de *shields* e circuitos externos.

- **GND** – pinos de referência, terra (0V).
- **VIN** – Se o Arduino for alimentado com 12V via conector de entrada, este pino terá os mesmos 12V. É uma alternativa para alimentar outros dispositivos que necessitem de mais de 5V.

Esses conectores devem ser usados com cuidado, pois cada componente tem um valor máximo de tensão, não podendo ser conectado a um valor de corrente superior ao suportado pela componente.

Para que qualquer placa eletrônica funcione (com o Arduino não é diferente), é preciso haver um “cérebro”, dispositivo que interpreta ou roda (como é falado entre os programadores) o código enviado para o Arduino. Observe a Figura 6, ela é uma montagem com duas imagens. A primeira é o microcontrolador utilizado no Arduino UNO e a segunda é o esquema de ligação de cada pino do microcontrolador.

Figura 6 – Microcontrolador e pinagem ATmega328 usado no Arduino UNO



Fonte: Adaptado de Mota, 2017; Souza, 2013

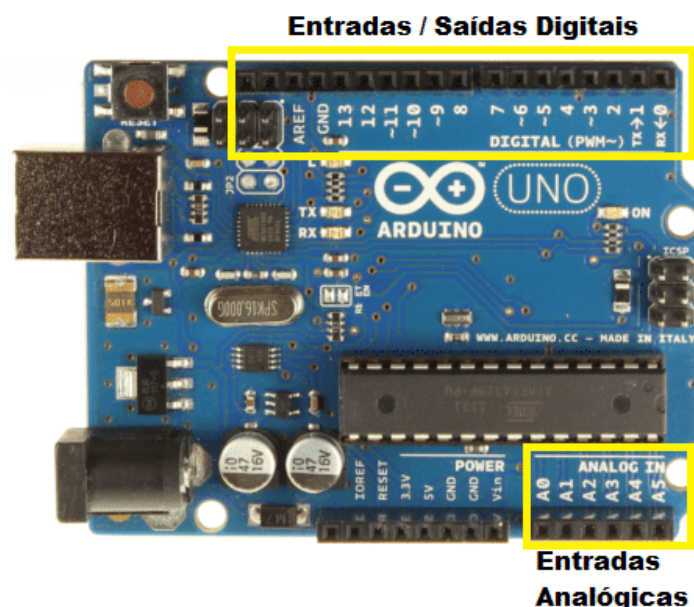
O microcontrolador (Figura 6) é o componente principal da placa do Arduino. Existem várias opções de marcas e modelos de microcontroladores, o modelo UNO, por exemplo, usa o microcontrolador ATmega328⁴.

TEMA 4 – DIGITAL X ANALÓGICO

Para estabelecer uma conexão com outros componentes, o Arduino dispõe de entradas e saídas digitais (I/O) e entradas analógicas. Observe a Figura 7.

⁴ Possui 32 KB de Flash (mas 512 bytes são utilizados pro *bootloader*), 2 KB de RAM e 1 KB de EEPROM.

Figura 7 – Entradas e saídas do Arduino UNO



Fonte: Souza, 2013.

As entradas e saídas também podem ser chamadas de “pinos”. Note, na Figura 7, que o Arduino UNO tem 14 pinos digitais (0 até 13) disponíveis para fazer conexões de entradas ou saídas digitais. Os pinos digitais operam com tensão de 5V e cada pino pode receber no máximo 40mA.

Ainda na Figura 7, alguns pinos (3,5,6,9,10 e 11) apresentam o símbolo ~ na frente da sua identificação. Esses pinos podem ser usados como saídas PWM⁵ de 8 bits por meio da função `analogWrite()`. Os pinos digitais só apresentam 2 estados, HIGH ou LOW (ligado ou desligado), ou seja, 0V ou 5V. Assim, só é possível utilizá-los em projetos do tipo “verificar se uma luz está ligada ou desligada”, “conferir se a janela está aberta ou fechada”, “observar se o botão está pressionado ou não”. Porém, em muitas situações, é preciso medir um valor de grandeza, e não somente saber se ele está ligado ou desligado. Para isso, são utilizadas as entradas analógicas.

O Arduino internamente só trabalha com dados digitais e tem um conversor do sinal analógico para um valor digital. Esse processo é conhecido como ADC, ou apenas conversor A/D, que quantifica o valor analógico conforme a quantidade de bits existentes na sua resolução e converte-o para um valor digital.

⁵ PWM, do inglês *Pulse Width Modulation*, é uma técnica utilizada por sistemas digitais para variação do valor médio de uma forma de onda periódica. A técnica consiste em manter a frequência de uma onda quadrada fixa e variar o tempo que o sinal fica em nível lógico alto. Esse tempo é chamado de *duty cycle*, ou seja, o ciclo ativo da forma de onda. Fonte: <<https://www.embarcados.com.br/pwm-do-arduino/>>

Observe, na Figura 7, que a placa do Arduino UNO apresenta 6 entradas analógicas (conversor analógico digital), nomeadas de A0 até A5.

Como exemplos de medições analógicas, podemos citar o velocímetro de um carro, considerado analógico, pois o ponteiro varia conforme o carro acelera ou freia.

Para fazer a manipulação da placa, o usuário deve tomar cuidado, pois, entre os pinos do microcontrolador e a barra de pinos, não há resistor que limite a corrente. Outra dica importante é que a parte traseira da placa, como observado na Figura 8, não apresenta qualquer tipo de isolamento, podendo gerar curto circuito dependendo do local em que a placa estiver. Por isso, o melhor local é sempre mesas de madeira, pois não conduzem eletricidade.

Figura 8 – Parte de trás do Arduino UNO



Fonte: Golubovy/Shutterstock.

Observando as imagens do Arduino, percebemos que não há botão para ligar ou desligar a placa. Então, para desligar o Arduino, basta retirar o cabo de alimentação (USB ou Jack). Ao lado do conector USB, na placa, há um botão *reset*, que serve para reiniciar a placa, apagando as suas programações.

TEMA 5 – ESTRUTURA DE PROGRAMAÇÃO

Para fazer a programação do Arduino, é utilizada a IDE. Como visto anteriormente, a IDE pode ser baixada diretamente do site oficial do Arduino ou mesmo ser utilizada a IDE on-line (link no mesmo site). Nesse ambiente, o usuário deve digitar os códigos e transferir a programação para o Arduino, podendo também fazer testes para encontrar possíveis erros. Lembrando que, para essa transferência, o Arduino deve estar conectado ao computador com o cabo USB.

Os passos para a programação do Arduino podem ser divididos como:

1. Abrir o IDE desejado (instalado no computador ou on-line);
2. Conectar o Arduino ao computador via cabo USB;
3. Desenvolver um programa (*sketch* de comandos);
4. Enviar o *sketch* (upload) para o Arduino;
5. O Arduino é reiniciado e o *sketch* começa a ser executado.

É importante salientar que o programa só é enviado para o Arduino quando não apresentar erro algum. O programador deve corrigir os erros, que podem ser de software (algo digitado errado ou até mesmo lógica errada) ou de hardware, quando alguma ligação foi feita de maneira errada na placa.

A seguir, será mostrada a estrutura básica da programação direta do Arduino. Dizemos que ela é direta pois o Arduino também pode ser programado via Scratch (programação por blocos). A Figura 9 representa uma IDE que está instalada no computador.

Figura 9 – Exemplo da IDE instalada no computador



A IDE representada na Figura 9 é dividida em 3 partes. Na parte superior, localiza-se o nome do *sketch*, juntamente com os menus (toolbar) que oferecem acesso a diversas funções. Abaixo do toolbar, há alguns ícones de atalho:

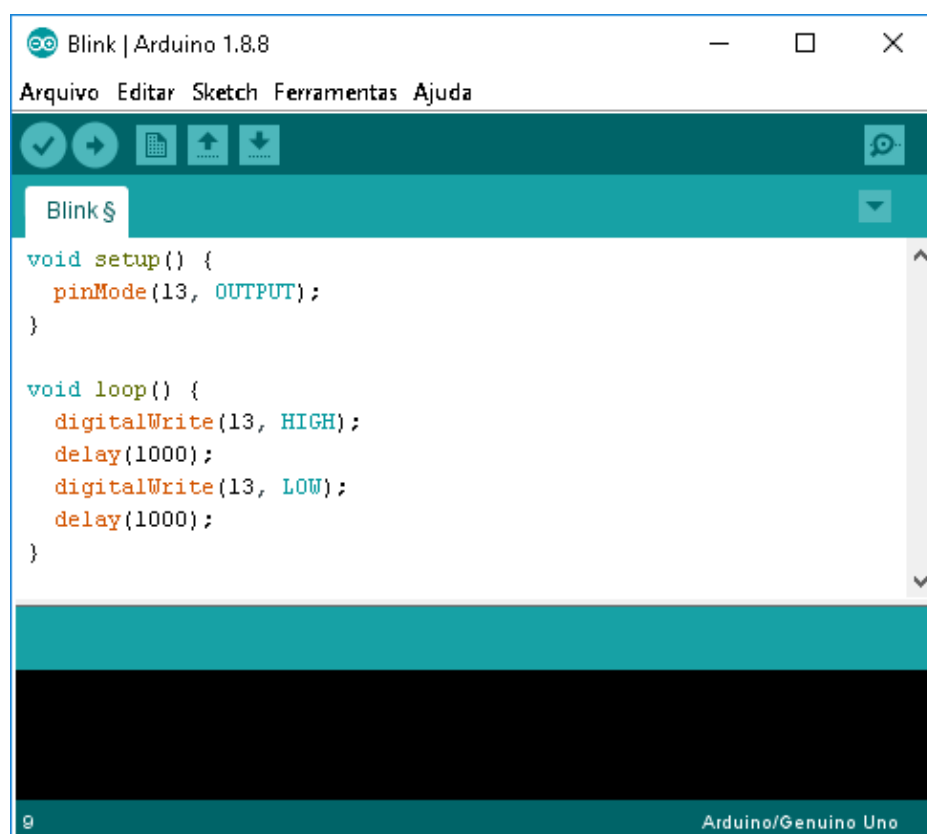
- Verificar → verifica se o código possui erros;
- Carregar → compila o código e envia o *sketch* para o Arduino;
- Novo → cria um novo *sketch*;
- Abrir → abre um *sketch* existente;
- Salvar → salva o *sketch* que está aberto;
- Monitor Serial → abre o painel do monitor serial, utilizado nos sensores analógicos.

A parte central da IDE (Figura 9) é o *sketch*, utilizada para fazer a programação. Por ser uma ferramenta open, o Arduino tem grande quantidade de exemplos já prontos que podem ser aproveitados e/ou adaptados. A programação é dividida em duas estruturas básicas: `setup()` e `loop()`.

- `setup()` → utilizada para as configurações iniciais do programa. Uma variável recebe um valor inicial. Mensagens para o usuário e saber se um pino vai ser utilizado para entrada ou saída são exemplos de configurações iniciais.
- `loop()` → os comandos inseridos nessa estrutura ficam se repetindo até que o Arduino receba (manual ou automático) algum comando para parar.

A própria IDE tem alguns exemplos já configurados, localizados no menu Arquivos na aba Exemplos. O exemplo mais didático é o Blink, que está representado na Figura 10.

Figura 10 – Exemplo Blink



Fonte: Adaptado de Thomsen, 2014.

A Figura 10 representa o exemplo Blink da IDE do Arduino. Para facilitar a visualização, retiramos os comentários do código. A Figura 11 é o mesmo exemplo, só que com uma explicação, linha a linha, em forma de comentário.

Figura 11 – Exemplo Blink com comentários

```
1 //Programa : Pisca Led Arduino  
2 //Autor : FILIPEFLOP  
3  
4 void setup()  
5 {  
6   //Define a porta do led como saída  
7   pinMode(13, OUTPUT);  
8 }  
9  
10 void loop()  
11 {  
12   //Acende o led  
13   digitalWrite(13, HIGH);  
14  
15   //Aguarda o intervalo especificado  
16   delay(1000);  
17  
18   //Apaga o led  
19   digitalWrite(13, LOW);  
20  
21   //Aguarda o intervalo especificado  
22   delay(1000);  
23 }
```

Fonte: Thomsen, 2014.

Analisando a Figura 11, nota-se que as duas primeiras linhas são apenas informações como nome do programa e autoria. Todas as linhas que apresentam o símbolo de // na frente são comentários e não são interpretadas pelo compilador, ou seja, são apenas informações para a pessoa que está programando.

Na linha 7 da Figura 11, está sendo definido o pino que será utilizado no Arduino e seu tipo. Os pinos podem ser de dois tipos: INPUT (entrada) ou OUTPUT (saída). No exemplo, foi definido o pino 13 como OUTPUT, isso significa que esse pino vai ser utilizado para saída e não para captação de dados ou leitura de algum sensor.

Como o pino 13 é um pino digital, na linha 13 do exemplo, ele recebe o valor HIGH (que significa ligado ou 1) e, na linha 19, o valor LOW (que significa desligado ou 0).

A função delay(), utilizada nas linhas 16 e 22, serve para definir o tempo de execução entre os comandos. Delay(1000) quer dizer 1000 milissegundos, o que representa 1 segundo.

Interpretando o programa da Figura 11, o *sketch* vai ligar o que estiver plugado no pino 13, esperar 1 segundo (1000 ms), desligar o pino 13 e esperar 1 segundo. Isso vai se repetir infinitamente, pois está programado dentro do loop() e sem qualquer programação para parar.

A terceira e última parte da IDE (Figura 9) é utilizada para mensagens e verificações de erros. O programador não vai informar código ou comando nessa região, mas ela é de extrema importância, pois retorna as mensagens de erros, tanto de programação, quanto de conexão.

REFERÊNCIAS

GOGONI, R. O que é internet das coisas? **Tecnoblog**, 2018. Disponível em: <<https://tecnoblog.net/263907/o-que-e-internet-das-coisas/>>. Acesso em: 19 mar. 2019.

MOTA, A. O que é Arduino e como funciona? **Vida de Silício**, 14 maio 2017. Disponível em: <<https://portal.vidadesilicio.com.br/o-que-e-arduino-e-como-funciona/>>. Acesso em: 19 mar. 2019.

SOUZA, F. Arduino Uno. **Embarcados**, 29 nov. 2013. Disponível em: <<https://www.embarcados.com.br/arduino-uno/>>. Acesso em: 19 mar. 2019.

MASSIMO Banzi: como o Arduino está tornando a imaginação Open-Source. **TED**, jun. 2012. Disponível em: <https://www.ted.com/talks/massimo_banzi_how_arduino_is_open_sourcing_imagination?language=pt-br>. Acesso em: 19 mar. 2019.

THOMSEN, A. O que é Arduino? **Filipe Flop**, 2 set. 2014. Disponível em: <<https://www.filipeflop.com/blog/o-que-e-arduino/>>. Acesso em: 19 mar. 2019.

_____. Qual Arduino comprar? Conheça os tipos de Arduino. **Filipe Flop**, 3 fev. 2014. Disponível em: <<https://www.filipeflop.com/blog/tipos-de-arduino-qual-comprar/>>. Acesso em: 19 mar. 2019.